Hisaab — Hackathon Master Plan

FINAL · RC3 Mobile App Development · Shared Living App Challenge · Node.js + PostgreSQL + Kotlin/Compose · built with AI Studio · 5 Nov 2026

0. Confirmed with the moderator

- **AI-assisted coding is fully allowed.** The oral defense (6 marks) still happens, so every member must be able to open and explain their own modules (Phase 12).
- **Real authentication is allowed** as a deviation from the PDF: register, login, sessions, household invites. The PDF's fixed Ali / Ayesha / Bilal become demo accounts created by the seed script.
- **Either sign convention for net balance is fine.** We use Net = Paid − Owed + Settlements Sent − Settlements Received, which matches the PDF's own worked example (Ayesha −450 → −200 after paying Ali 250). The sum is 0.00 either way.
- **Time:** the PDF says 5 hours with the surprise at hour 3.5; you have 3. This plan is time-boxed to 180 min, the schema is pre-wired for all four surprise options, and the last 30 min are reserved for the surprise plus freeze.
- **Scope decision:** one household per deployment (a single household row; members join by invite code). Multi-household is a stretch item. This keeps the benchmark member IDs M01/M02/M03 intact.
- **Home screen 'active member selector'** from the PDF becomes a **View as** selector: it defaults to the logged-in member, and picking another member shows that person's balance card (read-only). Actions always run as the logged-in member.

1. Where the 100 marks live

Rubric	Marks	What wins it	Phases
Functionality	50	REST API 15, checkout 10, expense and split logic 15, surprise 10	1-5, 8, 10
Design	20	Hierarchy, spacing, navigation, PKR labels, status indicators, polished auth flow	6-8
Code quality	10	Layers, naming, state management, clean API client	2, 6
UX and verification	20	Validation, spinners, double-tap prevention, oral defense 6, demo 4	9, 11, 12

Priority rule: money correctness > required screens > auth > resilience > polish > stretch. If behind, cut in this order: stretch items, dark mode, history lists, change-password UI. **Never cut:** money correctness, validation, persistence, double-submit protection, token-protected endpoints.

2. Locked decisions (do not re-debate)

- **Money** is integer paisa (BIGINT) in the DB and engine, never floats. The API sends and receives JSON numbers with at most 2 decimals (333.34), converted at the edge only. The client does math in BigDecimal/Long paisa; Double is transport only.
- **IDs:** members M01, M02, M03... (text PK, assigned in join order). Ascending ID order drives remainder distribution.
- **Time zone** Asia/Karachi (UTC+5, no DST). A month is the calendar month of expense_date.
- **Budget:** PKR 5,000 default in settings. Status: On Track below 80%, Near Limit 80% to under 100%, Over Budget at 100% or more (84% = Near Limit as in the PDF).
- **Categories:** Groceries, Utilities, Cleaning, Entertainment, Dining, Other. **Units:** pack, dozen, bottle, bag.
- **Errors:** envelope {error:{code,message,fields?}}. VALIDATION_ERROR 400, UNAUTHENTICATED 401, FORBIDDEN 403, NOT_FOUND 404, CONFLICT 409, BUSINESS_RULE 422, RATE_LIMITED 429, INTERNAL 500.
- **Auth:** email + password. Passwords hashed with bcryptjs (cost 12, pure JS so no native build risk). Access token = JWT HS256, 15 min. Refresh token = 32 random bytes, stored only as a SHA-256 hash, 30 days, rotated on every use with reuse detection.
- **Roles:** admin (the first registered member, M01) and member. All endpoints except /auth/register, /auth/login, /auth/refresh and /health require a Bearer token.
- **Concurrency:** every money-writing transaction takes pg_advisory_xact_lock. POST endpoints accept an Idempotency-Key header. Balances are derived from the ledger on every read, never stored.
- **Android:** Kotlin, Jetpack Compose, Material 3, MVVM + StateFlow, Retrofit + OkHttp + kotlinx.serialization, Navigation Compose, DataStore, manual DI via an AppContainer (Hilt adds build risk in 3 hours). minSdk 26. Use the version catalog Android Studio generates; never let AI Studio invent dependency versions.

- **Backend:** Node LTS, Express, pg, zod, jsonwebtoken, bcryptjs, helmet, express-rate-limit, pino, Jest + supertest.

3. API contract (freeze at minute 10, save as docs/CONTRACT.md)

Auth and household (no Bearer needed for the first three)

Endpoint	Request	Response
POST /api/auth/register	{name,email,password,invite_code?}. The very first account needs no code and becomes admin M01; later accounts need the household invite code	201 {user,access_token,refresh_token,expires_in}
POST /api/auth/login	{email,password}	200 same shape as register
POST /api/auth/refresh	{refresh_token}	200 new token pair (old one revoked)
POST /api/auth/logout	{refresh_token}	204, token family revoked
GET /api/auth/me	-	{user:{id,member_id,name,email,role}}
POST /api/auth/password	{current_password,new_password}	204, all other sessions revoked
GET /api/household	-	{name,member_count,invite_code (admin only)}
POST /api/household/invite/rotate	- (admin only)	200 {invite_code}

Business endpoints (all require Bearer)

Endpoint	Request	Response
GET /api/members	-	[{id,name}]
GET /api/shopping?status=	optional pending/bought	[{id,title,unit,requested_qty,status,bought_at}]
POST /api/shopping	{title,unit,requested_qty}	201 item
POST /api/shopping/checkout	{item_ids[],total_paid,payer_id,split_members[]}	201 {expense,items}
POST /api/expenses	{title,category,amount,payer_id,split_type: EQUAL or CUSTOM,splits[],expense_date?}	201 expense with computed shares
GET /api/balances	-	{balances:[{member_id,name,paid,owed,sent,received,net}],sum}
POST /api/settle	{sender_id,recipient_id,amount}	201 {settlement,balances}
GET /api/expenses?month=	-	expenses newest first, with shares
GET /api/settlements	-	settlements newest first
GET /api/budget?month=YYYY-MM	-	{month,monthly_budget,total_spent,remaining,percent_used,status}
GET /api/health	- (public)	{status,db}
GET /api/balances/simplify	-	[{from,to,amount}] (surprise opt 1)
GET /api/budget/categories	-	per-category totals and flags (opt 3)
POST /api/expenses/recurring/apply	-	{created,skipped} (opt 4)

splits: EQUAL takes [{member_id}] (a plain id list is also accepted). CUSTOM takes [{member_id,amount}] and must sum to amount exactly. **Authorization rules:** created_by always comes from the token, never the body. Settlements can be recorded by the sender, the recipient or an admin (so a third party cannot fabricate payments; in the demo Ali is admin). Invite rotation is admin only.

4. Timeline (180 min, three tracks)

Roles: **A** Backend + DB + auth. **B** Android core + auth screens + Home + Money. **C** Money engine + Shopping screen + test scripts.

Time	A	B	C
0:00-0:15	Phase 0 (all together)	Phase 0	Phase 0

Time	A	B	C
0:15-0:35	Ph 1 Database	Ph 6 Android foundation	Ph 4 Money engine and tests
0:35-1:05	Ph 2 server skeleton, Ph 3 auth backend	Ph 6 cont., Ph 7 auth screens	Ph 4 finish, client Money util
1:05-1:40	Ph 5 business endpoints	Ph 8.1 Home, 8.3 Money	Ph 8.2 Shopping
1:40-2:05	Ph 9 integration and resilience (all)		
2:05-2:30	Ph 11 verification passes A and B (all)		
2:30-2:55	Ph 10 surprise (A backend, B/C UI)		
2:55-3:00	Ph 12 freeze, APK, rehearsal		

Phase 0 — Setup and contract (0:00-0:15)

- **S0.1 Repo.** Monorepo hisaab/ with /server, /android, /docs. .gitignore covers node_modules, .env, build, local.properties, *.keystore. Commit .env.example (with a JWT_SECRET placeholder). Branch per person, merge after every seam.
- **S0.2 Smoke test.** Node, createdb hisaab, emulator booted, adb devices. Prove a hello-world round trip (app, server, DB) before any feature. The emulator reaches the host at 10.0.2.2:3000; a physical phone uses adb reverse tcp:3000 tcp:3000 and 127.0.0.1.
- **S0.3 Contract freeze.** Section 3 goes into docs/CONTRACT.md. Any change is announced to all three.
- **S0.4 AI Studio protocol.** See the last section.

Phase 1 — Database (A)

- **S1.1 Schema** (all money BIGINT paisa):
- household(id=1, name, invite_code); settings(key PK, value) seeded with monthly_budget_paisa = 500000.
- members(id M01.., name, email UNIQUE lowercase, password_hash, role CHECK admin/member, failed_attempts, locked_until, created_at).
- refresh_tokens(id, member_id FK, family_id, token_hash UNIQUE, expires_at, revoked_at, replaced_by, created_at).
- shopping_items(id, title VARCHAR(80), unit CHECK 4 units, requested_qty CHECK > 0, status CHECK pending/bought, expense_id FK null, created_by, created_at, bought_at).
- expenses(id, title VARCHAR(80), category CHECK, amount_paisa CHECK > 0, payer_id FK, split_type CHECK, expense_date DATE, source CHECK manual/checkout/recurring, created_by FK, **subtotal_paisa, tax_percent NUMERIC(5,2), discount_paisa, is_recurring** pre-wired for surprises, created_at).
- expense_splits(expense_id FK cascade, member_id FK, share_paisa CHECK >= 0, PK both); settlements(id, sender_id, recipient_id, amount_paisa CHECK > 0, CHECK sender <> recipient, recorded_by FK, created_at).
- recurring_templates(id, title, category, amount_paisa, payer_id, split_type, splits_json, active); recurring_runs(template_id, month CHAR(7), expense_id, UNIQUE(template_id, month)); idempotency_keys(key, endpoint, member_id, status, response JSONB, created_at, PK(key, endpoint)).
- **S1.2 Integrity.** FKs, CHECKs, indexes on expense_date, payer_id, status, refresh_tokens(token_hash). SUM(shares) = amount is enforced in the service and re-checked by a verification query.
- **S1.3 Migrations and seeds.** Numbered SQL files applied in a transaction, tracked in schema_migrations. Scripts: npm run migrate; seed (household, budget, demo accounts Ali M01 admin, Ayesha M02, Bilal M03 with a documented dev-only password); seed:demo (loads the 4 benchmark expenses through the service layer); reset:demo (refuses when NODE_ENV=production).
- **S1.4 Least privilege.** The app connects with its own DB role, not the superuser. **S1.5 Persistence proof:** restart the server, then Postgres; data intact.

Phase 2 — Backend foundation (A)

- **S2.1 Layers.** Route, controller, service, repository. Folders: modules/{auth,household,members,shopping,expenses,balances,settlements,budget}, engine/, middleware/, db/, utils/. Controllers hold no logic; services hold no SQL.
- **S2.2 Config.** dotenv validated at boot (fail fast; JWT_SECRET at least 32 chars), pool max 10 with timeouts and statement_timeout, no secrets in git.
- **S2.3 Validation.** zod per route, strict (unknown keys rejected), trimmed strings, title 1-80 chars and not whitespace-only. Amounts: positive, at most 2 decimals, at most 10,000,000, no NaN, Infinity or exponent forms.

- **S2.4 Errors.** AppError classes, one central handler, no stack traces to clients, malformed JSON gives 400, unknown route 404, request-id on every log line. Logs never contain passwords or tokens.
- **S2.5 Hardening.** helmet, 100kb body limit, global rate limit (120/min per IP, 30/min on POST), x-powered-by off, parameterized queries only.
- **S2.6 Idempotency middleware.** Same key, endpoint and member replays the stored response with no second write. **S2.7 Lifecycle:** /api/health pings the DB; graceful shutdown closes the pool.

Phase 3 — Authentication and authorization, backend (A)

- **S3.1 Register.** Normalize email (trim, lowercase). Password 8-72 chars (bcrypt limit), rejected if it equals the email or is in a short common-password list. First account ever: creates admin M01 with no code. Later accounts need the invite code (compare in constant time); wrong code gives 422. Duplicate email gives 409. Member cap 10. Returns tokens so the app logs in immediately.
- **S3.2 Login.** Generic error 'Invalid email or password' for both unknown email and wrong password; compare against a dummy hash when the email is unknown so timing does not leak which emails exist. Five consecutive failures lock the account for 15 min (429 with retry_after). Success resets the counter.
- **S3.3 Tokens.** Access JWT: sub = member id, role, 15 min, algorithm pinned to HS256 on verify. Refresh: random 32 bytes, SHA-256 stored, 30 days, new family per login. Refresh rotates: old token revoked and linked via replaced_by. If a revoked token is presented again, the whole family is revoked (theft detection). Logout revokes the family. Password change revokes every other family.
- **S3.4 requireAuth middleware.** Verifies the Bearer token and loads the member. A deleted or locked member gets 401. Applied to every route except register, login, refresh and health. **S3.5 requireAdmin** guards invite rotation.
- **S3.6 Authorization rules.** created_by and recorded_by come from the token. Settlement caller must be sender, recipient or admin (else 403). Everything is scoped to the single household; no endpoint returns emails or password data to other members.
- **S3.7 Auth rate limits.** /auth/login and /auth/register: 10/min per IP, stacked on the per-account lock. **S3.8 Tests:** every row of the auth negative table in Phase 11.
- **Out of scope (say so if asked):** email verification and password reset need an email service. Admin-initiated reset is a stretch item.

Phase 4 — Money engine (C; pure functions, tests written first)

- **S4.1 money.js.** Strict toPaisa (string or number, reject more than 2 decimals) and fromPaisa to a number with 2 decimals.
- **S4.2 splitEqual(amount, ids).** Sort ids ascending, base = floor(a/n), r = a mod n, the first r members get +1 paisa. Tests: 100000/3 gives [33334, 33333, 33333]; 1 paisa across 3 gives [1,0,0]; n = 1; payer not in the split.
- **S4.3 splitCustom.** Reject duplicates, unknown members, negatives and any sum mismatch (message states the difference).
- **S4.4 computeBalances.** Net = paid − owed + sent − received. Assert the sum is exactly 0; if not, throw INTERNAL and log loudly.
- **S4.5 validateSettlement.** Amount > 0, sender differs from recipient, both exist, amount <= sender's debt (max(0, −net)) and <= recipient's credit (max(0, net)). Specific messages, e.g. 'You only owe PKR 200.00'. No debt gives 'Nothing to settle'.
- **S4.6 simplify.** Greedy: largest debtor pays largest creditor, ties go to the lower id, repeat until zero. At most n−1 transfers.
- **S4.7 Benchmark test.** Four PDF expenses give total 4,200.00; nets +250 / −450 / +200; sum 0.00; settle 250 then 200; spending still 4,200.00; a further settlement is rejected. Add a randomized test: shares always sum to the amount, balances always sum to 0.

Phase 5 — Business endpoints (A)

- **S5.1 Members.** GET, ordered by id; id and name only.
- **S5.2 Shopping.** GET with filter. POST validates unit enum and qty 1-999. A case-insensitive duplicate of a pending title returns 409 ('already on the list'), which prevents the 'logged twice' problem from the scenario.
- **S5.3 Checkout** (one transaction, advisory lock). SELECT items FOR UPDATE; ids unique, existing and pending, else 409 naming the offenders. total_paid > 0, payer exists, split_members non-empty, unique, existing. Equal split. Insert the expense (category Groceries, title like 'Groceries: Milk, Rice' cut to 80 chars, source checkout, date today in Karachi, created_by from token). Mark items bought and linked. The payer need not be in the

split.

- **S5.4 Manual expense.** Full validation. Date not in the future and not older than 1 year. EQUAL or CUSTOM. Atomic insert of expense and shares.
- **S5.5 Balances and budget.** Balances derived each call. Budget total excludes settlements, guards a zero budget, validates the month param.
- **S5.6 Settle.** Transaction, lock, recompute balances inside the lock, authorization rule, validateSettlement, insert, return the new balances. Two parallel settles of the same debt: one succeeds, one gets 422.
- **S5.7 History** endpoints. **S5.8 Contract tests:** supertest for every row of the Phase 11 tables.

Phase 6 — Android foundation (B)

- **S6.1 Project.** Package com.hisaab.app, minSdk 26, Compose, edge-to-edge, rotation supported through ViewModel state, allowBackup=false, R8 on in release, adaptive icon.
- **S6.2 Networking.** Retrofit interface mirrors the contract. OkHttp: connect 5s, read 15s, logging in debug only. A sealed ApiResult maps HTTP errors (parsing the error envelope), timeouts and IOException to friendly messages.
- **S6.3 Session layer.** SessionManager holds tokens, encrypted with an Android Keystore AES-GCM key and kept in DataStore (EncryptedSharedPreferences is deprecated, avoid it). An AuthInterceptor adds the Bearer header. An OkHttp Authenticator handles 401: one mutex-guarded refresh, retry once; if refresh fails, clear the session and route to Login with 'Session expired'. Backup rules exclude the token store.
- **S6.4 Architecture.** data/{remote,repository,model}, ui/{theme,components,auth,home,shopping,money,profile}, util/{Money,Dates}. Each screen has Screen + ViewModel + sealed UiState (Loading / Content / Error). One-shot events (snackbars) through SharedFlow. AppContainer for DI.
- **S6.5 Design system.** Teal #0F766E primary, amber #F59E0B accent and warning, green for credit, red for debt; light and dark; 4/8/16/24dp spacing; 16dp corners. Components: MoneyText ('PKR 4,200.00', color-coded), StatusChip, MemberAvatar (initials), LoadingButton, PasswordField (show/hide), ErrorBanner, EmptyState, SkeletonCard, ConfirmDialog.
- **S6.6 Navigation graph.** Auth graph (Welcome, Login, Register) and Main graph (bottom bar: Home, Shopping, Money; Profile from the top bar). Start destination decided by the splash. Per-tab state saved, sane back stack, imePadding.
- **S6.7 Connectivity and dev networking.** ConnectivityManager callback drives a global 'No connection' banner and auto-refresh on reconnect. BASE_URL through buildConfigField; network_security_config allows cleartext in debug only.

Phase 7 — Android auth and onboarding (B)

- **S7.1 Splash.** SplashScreen API (core-splashscreen) with the brand icon. Held only until the stored session is checked (valid access token, or a silent refresh), capped at 1.5s with no fake delay. Result: Main graph, Login, or an offline state with Retry if the server is unreachable (never a blank screen).
- **S7.2 Welcome.** Brand mark, one line of value ('Split it. Settle it. Done.'), two buttons: Sign in, Create account.
- **S7.3 Login.** Email and password fields with correct keyboard types, IME actions (Next, then Done submits), show/hide password, autofill semantics for username and password, inline field errors, LoadingButton, friendly messages for 401 ('Invalid email or password'), 429 ('Too many attempts, try again in N minutes') and network loss. Submit disabled while invalid or in flight.
- **S7.4 Register.** Name, email, password with a live strength hint, and a choice: 'Create household' (only offered when the server says no household members exist yet) or 'Join with invite code'. Server errors (409 email taken, 422 bad code) land on the right field.
- **S7.5 Profile.** Name, email, role, household name, invite code with Copy and Share (admin sees it and can rotate it), change password, Sign out (confirm dialog; calls /auth/logout and clears local tokens even if the call fails), app version.
- **S7.6 Session events.** Forced logout (revoked or expired refresh token) navigates to Login with a snackbar. Draft form values are never persisted across logout.
- **S7.7 Stretch:** biometric unlock on app resume (BiometricPrompt) behind a Profile toggle.

Phase 8 — Core screens (B and C)

- **S8.1 Home.** 'View as' chips with avatars (default: me); switching updates the balance card at once. Spend card: 'PKR 4,200.00 of PKR 5,000.00', progress bar clamped to 0-100%, colored by status, status chip, remaining or over text. Net card copy exactly: 'You are owed PKR 250.00', 'You owe PKR 450.00', or 'You are all

settled up'. Pending-items card that opens Shopping. Pull to refresh. The data loads run in parallel and fail independently, each card with its own retry.

- **S8.2 Shopping.** Filter chips (All, Pending, Bought); rows with status tags; bought rows are not selectable. FAB opens an Add Item sheet: title, unit chips (pack, dozen, bottle, bag), qty stepper 1-999, inline validation. A sticky 'Checkout (n)' bar opens the Checkout sheet: items summary, total paid (decimal keypad, live format), payer (default me), split-between chips (default all, at least 1) with a live per-person preview using the same remainder rule (Kotlin port with unit tests), and a LoadingButton disabled while invalid or in flight. On 409, refresh the list and show 'Some items were already bought'.
- **S8.3 Money tab.** (a) Balance table: member, paid, owed, net colored, plus a total row showing PKR 0.00. (b) Add Expense: title, category dropdown, date picker (default today, no future dates), amount, payer (default me), Equal/Custom toggle; Custom shows per-member fields with a running 'Remaining PKR x' and blocks submit on mismatch. (c) Settle Up dialog: sender, recipient (cannot equal sender), amount, helper text 'Ayesha owes PKR 200.00', a 'Settle full amount' shortcut, client pre-validation mirroring the server (including the who-may-record rule), server errors shown inline. (d) Recent activity list of expenses and settlements.
- **S8.4 States and accessibility.** Empty states, huge amounts (9,999,999.99 without breaking layout), long names with ellipsis, font scale 200%, dark mode, TalkBack labels, 48dp touch targets, AA contrast.
- **S8.5 Strings and formatting.** All text in strings.xml. One Money formatter and one date formatter app-wide.

Phase 9 — Integration and resilience (all; Q4 = 10 marks)

- **S9.1 Loading.** Skeletons for loads; an in-button spinner for submits.
- **S9.2 Double submit.** ViewModel in-flight flag, disabled button, an Idempotency-Key generated when the form opens (reused on retry, rotated after success), plus the server replay. Test: tap Confirm five times fast, get one expense.
- **S9.3 Errors.** Envelope to readable message; field errors shown on the field; network drop gives a banner with Retry; timeout gives 'Server is taking too long'; 5xx gives a generic message.
- **S9.4 Persistence.** Force-stop the app and reopen: still logged in and all data returns. Stop the server mid-use: offline state, no crash. Restart server and Postgres: data intact, session still valid.
- **S9.5 State survival.** Form drafts in SavedStateHandle; rotation and process death do not lose input. Pressing back during a submit lets the request finish and the result is still handled.

Phase 10 — Surprise readiness (pre-wired now, executed when released)

First restate the released text in 3 lines, then build. Afterwards re-run the benchmark script so nothing regressed.

Option	Backend	UI	Est.
1 Smart Settle (default pick)	/api/balances/simplify using the existing simplify()	'Smart Settle Suggestions' button, cards like 'Ayesha pays Ali PKR 250.00', One-Tap Settle calling POST /api/settle, refresh after; a stale suggestion (422) triggers a refetch. Respect the who-may-record rule	25 min
2 Tax and discount	checkout accepts tax_percent (0-100, 2dp) and discount_amount. Final = round-half-up(items_total x (100 + tax) / 100) minus discount, must stay above 0. total_paid is then the items subtotal; store subtotal, tax, discount	Two new fields with live net total, shown as a breakdown before confirm, using the same rounding as the server	30 min
3 Category warnings	/api/budget/categories: totals per category for the month, flag when strictly above 40% of the budget (2,000.00)	Category list on Home, prominent warning badge. With the benchmark data Utilities = 2,520.00 is flagged, which is a good demo	30 min
4 Recurring	is_recurring stores a template; apply creates one expense per active template for this month, idempotent through UNIQUE(template_id, month); returns created and skipped	Checkbox 'Mark as Monthly Recurring Bill', 'Generate Monthly Recurring Bills' button on Home	30 min

Phase 11 — Verification (two full passes)

Pass A — automated

npm test; npm run verify:benchmark prints actual vs expected for every PDF number (4,200.00 spend, 800.00 remaining, 84%, Near Limit, +250 / −450 / +200, post-settlement zeros). Then the negative tests:

#	Business case	Expected
1	Expense amount 0 or negative; 10.005; '1e3'; NaN	400
2	CUSTOM splits sum 999.99 for 1000.00	400 with difference
3	Unknown payer or member; duplicate member in splits; empty split_members	400 with field
4	Title spaces-only or 81 chars; future expense_date; GET /api/budget?month=2026-13	400
5	Checkout with an already bought item	409
6	Checkout with duplicate item_ids	400
7	Same checkout twice with the same Idempotency-Key	same 201, one expense
8	Settle amount 0 or negative; settle to self	400
9	Settle above debt or above recipient credit; sender owes nothing	422
10	Two parallel settles of 250 against a 250 debt	one 201, one 422
11	Malformed JSON	400

#	Auth case	Expected
A1	Any business endpoint with no token, expired token, tampered token, or alg=none token	401
A2	Register with an existing email (any letter case)	409
A3	Register with a password under 8 or over 72 chars, or equal to the email	400
A4	Register as a later member with a wrong or missing invite code	422
A5	Login with a wrong password and with an unknown email	401, identical message
A6	Sixth consecutive failed login	429 with retry_after; correct password also blocked until the lock ends
A7	Refresh with a valid token	200, old token now revoked
A8	Re-use an already rotated refresh token	401 and the whole family is revoked
A9	Logout, then refresh with that token	401
A10	Member calls invite rotation	403
A11	Third party (not sender, recipient or admin) records a settlement	403
A12	Change password, then use an older session's refresh token	401
A13	Responses and logs checked for password hashes, tokens, other members' emails	none present

Pass B — device checklist

- Fresh install, splash, Welcome, Register (first user becomes admin), Home. Second device or account joins with the invite code.
- Sign out and sign in; force-stop and reopen stays logged in; an expired access token refreshes silently; a revoked refresh token lands on Login with a message.
- 'View as' switch updates the net card instantly. Add an item with each unit; an empty title is blocked.
- Select 2 items, checkout, both turn Bought, Home spend and pending count update. Double-tap Confirm: one expense only.
- Server stopped or airplane mode during submit: banner, no crash, Retry works. Restart server and Postgres: data present.
- Custom split mismatch blocked client-side; over-settlement blocked. Rotation, dark mode, font 200%, long title, 9,999,999.99.
- Benchmark reproduced end to end on the device.

Security checklist

Parameterized SQL only; no secrets in git; JWT_SECRET from env; passwords bcrypt-hashed and never logged; refresh tokens stored hashed; JWT algorithm pinned; helmet, rate limits and body limit; strict validation; no stack traces in responses; cleartext and HTTP logging debug-only; tokens encrypted at rest on device; allowBackup false; R8 in release; least-privilege DB role; npm audit reviewed.

Phase 12 — Demo, defense, packaging

- **S12.1 Demo script (5 min).** 1) Reset to the clean seed and sign in as Ali (admin). 2) Shopping: add 3 items, select 2, checkout with total paid, payer and split. 3) Log two shared bills (one equal, one custom). 4) Balances: show nets and the zero-sum row. 5) Settle Up: show a rejected over-settlement, then a valid one. 6) Home: budget bar and Near Limit status, then 'View as' Ayesha. 7) Surprise feature. 8) Kill and relaunch the app to show persistence, then sign out and in to show auth.
- **S12.2 Defense cards.** Each owner writes 5 lines per module as they finish. Likely questions: why integer paisa; how the remainder is distributed and why ascending id; how you prove the sum is 0 and which sign convention you chose; why a transaction plus advisory lock; what idempotency keys solve; why refresh tokens are hashed and rotated; why login errors are generic; how the Authenticator avoids refresh races; where state lives in the app and how UiState flows; how double-tap is prevented. Each person can open and explain their own files cold.
- **S12.3 Packaging.** Debug APK built and installed on the demo device; backup screenshots and a 30-second screen recording as fallback; npm start script; laptop checklist: Postgres running, port 3000 free, emulator warmed, adb reverse set (no dependence on venue Wi-Fi), charger.
- **Stretch (only with 20+ spare minutes):** biometric unlock, admin-initiated password reset, multi-household, Room read cache, PERCENT split, expense edit/delete with reversal, CSV export.

Edge-case master list

- **Money:** remainder paisa, 1 paisa across 3 members, payer outside the split, payer alone in the split, zero custom share, 9,999,999.99, settlement when all balances are zero, half-up rounding on tax.
- **Input:** emoji and RTL names, whitespace-only text, comma decimal separator on the keypad, leading zeros, pasted text with currency symbols, 3+ decimals, emails with capitals or trailing spaces, passwords with emoji (72-byte limit counts bytes).
- **Auth:** token expiry mid-request, two requests refreshing at once, clock skew, logout on one device while another is open, invite code rotated after being shared, last admin signing out, account locked while logged in elsewhere.
- **Concurrency:** double tap, two phones checking out the same item, retry after a timeout, parallel settles.
- **Time:** month rollover at midnight Karachi, first and last day of a month, backdated expenses.
- **Budget:** zero budget (no divide by zero), over 100% (bar clamped), exactly 80% and exactly 100%.
- **Client:** rotation, process death, back during submit, keyboard covering fields, slow network, server down at launch, bought items hidden from selection.
- **Environment:** emulator vs device networking, cleartext only in debug, Postgres not started, port in use.

Traceability — every PDF line mapped to a seam

PDF requirement	Seam
6 required endpoints, DB persistence	S5.1-5.6, S1.5
Home: month spend and progress bar, member selector, net card copy, pending count	S8.1 (selector becomes 'View as')
Shopping: status tags, add form with 4 units and qty, multi-select checkout	S8.2, S5.2-5.3
Expenses screen: title, category, date, amount, payer, split strategy	S8.3(b), S5.4
Live balance table and Settle Up dialog	S8.3(a)(c), S5.5-5.6
Equal split with remainder in ascending id; 333.34 / 333.33 / 333.33	S4.2
Net balance equation, sum always 0.00	S4.4 (sign convention approved)
Settlement rules: non-positive, self, over-settlement; spending unchanged	S4.5, S5.6
Spinners, no double submit, error toasts, persistence across restarts	Phase 9
Hour 3.5 surprise	Phase 10
5-minute demo and individual oral defense	Phase 12
Benchmark: 4,200 / 800 remaining / 84% Near Limit / +250 −450 +200 / settle to zero	S4.7, Phase 11
Tab or drawer navigation, PKR labels, status indicators	S6.6, S6.5
Code quality: route modularization, state management, clean API client	S2.1, S6.2-6.4
Added with moderator approval: register, login, sessions, household invites	Phases 3 and 7

AI Studio protocol

1. One seam per prompt. Build and run after every seam; never stack unbuilt seams. Commit after each.

2. Every prompt starts with the master context block below, then the frozen contract, then the seam text copied from this plan.
3. Ask for complete files with their paths. Paste compile errors back verbatim.
4. Forbid contract changes and new dependencies without asking. Pin versions from the version catalog.
5. After each phase, one review prompt: 'List violations of the rules above, missing validations, and edge cases from this list.' Iterate twice, then move on.
6. Read what it generated. You will be asked to explain it.

```
MASTER CONTEXT
App: Hisaab, shared household expenses for roommates in Lahore. Currency PKR.
Demo accounts: Ali M01 (admin), Ayesha M02, Bilal M03.
Backend: Node.js + Express + PostgreSQL (pg), layers route/controller/service/repository, zod validation.
Auth: email+password, bcryptjs, JWT access 15min, rotating hashed refresh tokens, roles admin/member.
Android: Kotlin, Jetpack Compose, Material 3, MVVM + StateFlow, Retrofit + kotlinx.serialization, manual DI.
Rules: money is integer paisa in DB and engine, never float. Equal-split remainder goes 1 paisa each
to members in ascending id order. Net = paid - owed + sent - received; all nets sum to 0.
Errors use {error:{code,message,fields}}. All strings in strings.xml. No hardcoded secrets.
Do not change the API contract. Output complete files with paths.
```

Name and logo prompt

Name: Hisaab (Urdu for 'accounts', as in hisaab-kitaab). Repo: **hisaab**.

Logo prompt for Google Flow: Minimal modern mobile app icon for 'Hisaab', a shared-household money app. A rounded-square icon containing a simple house silhouette whose body is a coin split into three equal segments, one for each roommate. Deep teal (#0F766E) and warm amber (#F59E0B) on an off-white background. Flat vector style, clean geometry, thick strokes, centered, 1:1, high contrast, legible at 48px. No text, no gradients, no photorealism, no extra decoration.